

CHAPITRE IX

GESTION DES TRANSACTIONS

I. INTRODUCTION

Cette unité traite de diverses instructions de traitement des **transactions** telles que **Démarrer transaction**, **Valider** et **Annulation**. Il décrit les instructions qui ne peuvent pas être annulées après l'exécution d'une transaction. Il décrit également les instructions qui entraînent une validation implicite après l'exécution.

Il traite de l'utilisation des points de sauvegarde dans une transaction et de leur incidence ou de leur utilisation pendant le traitement de la transaction.

OBJECTIFS

Après avoir étudié cette unité, vous devriez être capable de:

- Discuter de l'importance des instructions **START TRANSACTION**, **COMMIT** et **ROLLBACK**.
- Décrire les déclarations qui ne peuvent pas revenir en arrière
- Décrire les instructions qui provoquent une validation implicite avec des exemples pertinents
- Expliquer l'importance des instructions **SAVEPOINT** et **ROLLBACK to SAVEPOINT**
- Montrer le processus impliqué dans le **VERROUILLAGE** et le **DEVERROUILLAGE** des tables avec des exemples
- Expliquer l'utilisation de la syntaxe **SET TRANSACTION**

II. SYNTAXE START TRANSACTION, COMMIT ET ROLLBACK

MySQL prend en charge les transactions locales (au sein d'une session client donnée) via des instructions telles que **SET AUTOCOMMIT**, **START TRANSACTION**, **COMMIT** et **ROLLBACK**. La prise en charge des transactions XA permet à **MySQL** de participer également aux transactions distribuées.

```
START TRANSACTION [WITH CONSISTENT SNAPSHOT] | BEGIN  
[WORK]  
COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]  
ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]  
SET autocommit = {0 | 1}
```

- ❖ L'instruction **START TRANSACTION** ou **BEGIN** lance une nouvelle transaction.
- ❖ **COMMIT** valide la transaction en cours, rendant ses modifications permanentes.
- ❖ **ROLLBACK** annule la transaction en cours et annule ses modifications.
- ❖ L'instruction **SET AUTOCOMMIT** désactive ou active le mode **AUTOCOMMIT** par défaut pour la session en cours.
- ❖ Le mot-clé **WORK** facultatif est pris en charge pour **COMMIT** et **ROLLBACK**, de même que les clauses **CHAIN** et **RELEASE**.
- ❖ **CHAIN** et **RELEASE** peuvent être utilisés pour un contrôle supplémentaire sur l'achèvement de la transaction.
- ❖ La valeur de la variable système **COMPLETION_TYPE** détermine le comportement d'achèvement par défaut.
- ❖ La clause **AND CHAIN** fait en sorte qu'une nouvelle transaction commence dès la fin de la transaction en cours et que la nouvelle transaction a le même niveau d'isolation que la transaction qui vient d'être terminée.
- ❖ La clause **RELEASE** amène le serveur à déconnecter la session client en cours après la fin de la transaction en cours. Le fait d'inclure le mot clé **NO** supprime l'achèvement de **CHAIN** ou de **RELEASE**, ce qui peut être utile si la variable système **COMPLETION_TYPE** est défini pour provoquer le chaînage ou l'achèvement de la publication par défaut.

NB :

1. Par défaut, MySQL s'exécute avec le mode **AUTOCOMMIT** activé. Cela signifie que dès que vous exécutez une instruction qui met à jour (modifie) une table, MySQL enregistre la mise à jour sur le disque pour la rendre permanente.

Pour désactiver le mode de validation automatique, utilisez l'instruction suivante:

```
SET autocommit=0;
```

```
mysql> set autocommit=0;  
Query OK, 0 rows affected (0.00 sec)
```

Et pour l'activer, on fait le contraire :

```
mysql> set autocommit=1;  
Query OK, 0 rows affected (0.00 sec)
```

2. Après avoir désactivé le mode **AUTOCOMMIT** en définissant la variable **AUTOCOMMIT** sur **zéro**, vous devez utiliser **COMMIT** pour stocker vos modifications sur disque ou **ROLLBACK** si vous souhaitez ignorer les modifications apportées depuis

le début de votre transaction. Si vous souhaitez désactiver le mode **AUTOCOMMIT** pour une seule série d'instructions, vous pouvez utiliser l'instruction **START TRANSACTION**:

Application :

Soit

```
mysql> desc aa;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)   | YES  |     | NULL    |       |
| nom   | varchar(30) | YES  |     | NULL    |       |
| salaire | varchar(20) | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> desc bb;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | YES  |     | NULL    |       |
| recaptulatif | varchar(30) | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.02 sec)
```

En remplissant, nous avons :

```
mysql> select * from aa;
```

id	nom	salaire
1	marcelin	2300000
2	okoy	18800000
3	adoua	20540000
4	modjouo	16400000
5	okombi	13000000
1	macelin	8900000
2	okoy	13000000
3	adoua	17000500
4	modjouo	25000350
5	okombo	12000000

```
10 rows in set (0.00 sec)
```



```
mysql> select * from bb;
```

id	recaptulatif
1	
2	
3	
4	
5	

```
5 rows in set (0.00 sec)
```

Nous allons maintenant élaborer un exemple de transaction :

Exécuter une ou des requêtes sur la base **ensp**: ?

```
START TRANSACTION;
SELECT @A:=SUM(salaire) FROM aa WHERE id=1;
SELECT @B:=SUM(salaire) FROM aa WHERE id=2;
SELECT @C:=SUM(salaire) FROM aa WHERE id=3;
SELECT @D:=SUM(salaire) FROM aa WHERE id=4;
SELECT @E:=SUM(salaire) FROM aa WHERE id=5;

UPDATE bb SET recaptulatif=@A WHERE id=1;
UPDATE bb SET recaptulatif=@B WHERE id=2;
UPDATE bb SET recaptulatif=@C WHERE id=3;
UPDATE bb SET recaptulatif=@D WHERE id=4;
UPDATE bb SET recaptulatif=@E WHERE id=5;

SELECT * FROM bb;
COMMIT;
```

[Délimiteur ;] ☒ Afficher à nouveau la requête après exécution

NB : Il faut toujours tout sélectionner avant exécution.

Après exécution, nous aurons ceci :

```
mysql> select * from bb;
+----+-----+
| id | recaptulatif |
+----+-----+
| 1  | 11200000    |
| 2  | 31800000    |
| 3  | 37540500    |
| 4  | 41400350    |
| 5  | 25000000    |
+----+-----+
5 rows in set (0.00 sec)
```

- ❖ Avec **START TRANSACTION**, la validation automatique reste **désactivée** jusqu'à la fin de la transaction avec **COMMIT** ou **ROLLBACK**. Le mode **AUTOCOMMIT** revient alors à son état précédent.
- ❖ **BEGIN** et **BEGIN WORK** peuvent être utilisés à la place de **START TRANSACTION** pour lancer une transaction.
- ❖ Commencer une transaction entraîne la validation de toute transaction en attente. Commencer une transaction entraîne également la libération des **verrous de table** acquis avec **LOCK TABLES**, comme si vous aviez exécuté **UNLOCK TABLES**.
- ❖ Le début d'une transaction ne libère pas un verrou de lecture global acquis avec **FLUSH TABLES WITH READ LOCK**.

START TRANSACTION a été ajouté dans **MySQL 4.0.11**. Il s'agit de la syntaxe SQL standard et constitue le moyen recommandé pour démarrer une transaction ad hoc. **BEGIN** et **BEGIN WORK** sont disponibles dans les versions **3.23.17** et **3.23.19** de **MySQL**, respectivement.

NB :

1. Notez que si vous n'utilisez pas de tables sécurisées pour les transactions, toutes les modifications sont stockées en une seule fois, quel que soit le statut du mode **AUTOCOMMIT**.
2. Si vous émettez une instruction **ROLLBACK** après avoir mis à jour une table non transactionnelle dans une transaction, un avertissement **ER_WARNING_NOT_COMPLETE_ROLLBACK** se produit. Les modifications apportées aux tables sécurisées pour les transactions seront annulées, mais pas celles apportées aux tables non sécurisées pour les transactions.

Si vous utilisez **START TRANSACTION** ou **SET AUTOCOMMIT = 0**, vous devez utiliser le **journal binaire MySQL pour les sauvegardes** au lieu du journal de mise à jour plus ancien. Les

transactions sont stockées dans le journal binaire dans un bloc, lors de **COMMIT**. Les transactions annulées ne sont pas enregistrées.

(**EXCEPTION**: les modifications apportées aux tables non transactionnelles ne peuvent pas être annulées. Si une transaction annulée inclut des modifications aux tables non transactionnelles, la transaction complète est consignée avec une instruction **ROLLBACK** à la fin pour garantir que les modifications apportées aux tables non transactionnelles les tables sont répliquées.)

Vous pouvez modifier le niveau d'isolation pour les transactions avec **SET TRANSACTION ISOLATION LEVEL.**

QUESTIONS D'AUTO-EVALUATION

1. Avec **START TRANSACTION**, **AUTOCOMMIT** reste ____ jusqu'à ce que vous mettiez fin à la transaction avec **COMMIT** ou **ROLLBACK**.

III. DECLARATIONS QUI NE PEUVENT PAS ETRE ANNULEES

Certaines déclarations ne peuvent pas être annulées. En général, elles incluent les instructions LDD (langage de définition de données), telles que celles qui créent ou suppriment des bases de données, ou celles qui créent, suppriment ou modifient des tables.

Vous devez concevoir vos transactions de manière à ne pas inclure de telles déclarations. Si vous émettez une instruction qui ne peut pas être annulée plus tôt dans une transaction et qu'une autre instruction échoue par la suite, l'effet complet de la transaction ne peut pas être annulé par l'émission d'une instruction **ROLLBACK**.

IV. DECLARATIONS CAUSANT UN ENGAGEMENT IMPLICITE

Les instructions suivantes mettent fin implicitement à une transaction (comme si vous aviez effectué un COMMIT avant d'exécuter la déclaration):

- ALTER TABLE
- COMMENCE
- CREER INDEX
- DROP DATABASE
- DROP INDEX
- DROP TABLE
- LOAD MASTER DATA
- TABLES DE VERROUILLAGE
- RENAME TABLE
- SET AUTOCOMMIT = 1
- START TRANSACTION

- TABLE TRUNCATE

UNLOCK TABLES met également fin à une transaction si des tables sont actuellement verrouillées.

Avant **MySQL 4.0.13**, **CREATE TABLE** met fin à une transaction si le journal de mise à jour binaire est activé.

- ❖ Les transactions ne peuvent pas être imbriquées (étroitement liées). Ceci est une conséquence de la **COMMIT** implicite effectuée pour toute transaction en cours lorsque vous émettez une instruction **START TRANSACTION** ou l'un de ses synonymes.
- ❖ **ALTER TABLE**, **CREATE TABLE** et **DROP TABLE** ne commettent pas de transaction si le mot clé **TEMPORARY** est utilisé. (Cela ne s'applique pas aux autres opérations sur des tables temporaires telles que CREATE INDEX, qui provoquent une validation.)
- ❖ Cependant, bien qu'aucune validation implicite ne se produise, l'instruction ne peut pas non plus être annulée. Par conséquent, l'utilisation de telles instructions va à l'encontre de l'atomicité de la transaction: **par exemple**, si vous utilisez **CREATE TEMPORARY TABLE**, puis annulez la transaction, la table reste en existence.
- ❖ L'instruction **CREATE TABLE** dans **INNODB** est traitée comme une transaction unique. Cela signifie qu'un utilisateur **ROLLBACK** n'annule pas les instructions **CREATE TABLE** effectuées par l'utilisateur au cours de cette transaction.

Depuis **MySQL 5.1.3**, **ALTER VIEW**, **CREATE TRIGGER**, **CREATE VIEW**, **DROP TRIGGER** et **DROP VIEW** provoquent une validation implicite.

À compter de **MySQL 5.1.15**, **CREATE TABLE ... SELECT** provoque une validation implicite avant et après l'exécution de l'instruction lorsque vous créez des tables non temporaires. (Aucune validation ne se produit pour **CREATE TEMPORARY TABLE ... SELECT**.)

Cela permet d'éviter un problème lors de la réplication, où la table pourrait être créée sur le maître après une restauration, mais ne pas être enregistrée dans le journal binaire et donc pas répliquée sur l'esclave.

Les instructions qui utilisent ou modifient implicitement les tables de la base de données mysql: à partir de **MySQL 5.1.3**, **CREATE USER**, **DROP USER** et **RENAME USER** provoquent une validation implicite. Depuis **MySQL 5.1.23**, les instructions **GRANT**, **REVOKE** et **SET PASSWORD** entraînent une validation implicite.

Déclarations de contrôle des transactions et de verrouillage: **BEGIN**, **TABLES DE VERROUILLAGE**, **SET AUTOCOMMIT = 1** (si la valeur n'est pas déjà 1), **START TRANSACTION**, **UNLOCK TABLES**.

UNLOCK TABLES ne valide une transaction que si des tables ont déjà été verrouillées avec LOCK TABLES. Cela ne se produit pas pour UNLOCK TABLES après FLUSH TABLES WITH READ LOCK, car cette dernière instruction n'acquiert pas de verrous de niveau table.

Les transactions ne peuvent pas être imbriquées. Ceci est une conséquence de la validation implicite effectuée pour toute transaction en cours lorsque vous émettez une instruction START TRANSACTION ou l'un de ses synonymes.

Les instructions qui entraînent une validation implicite ne peuvent pas être utilisées dans une **transaction XA** tant que la transaction est à l'état **ACTIVE**.

L'instruction **BEGIN** diffère de l'utilisation du mot clé **BEGIN** qui démarre une instruction composée **BEGIN ... END**. Ce dernier ne provoque pas de validation implicite.

Instructions de chargement de données: LOAD DATA INFILE. Avant MySQL 5.1.12, LOAD DATA INFILE provoquait une validation implicite pour tous les moteurs de stockage. Depuis MySQL 5.1.12, cela entraîne une validation implicite uniquement pour les tables utilisant le moteur de stockage NDB.

Instructions administratives: INDEX CACHE, INDEX DE CHARGEMENT DANS CACHE. Depuis MySQL 5.1.10, ANALYZE TABLE, CHECK TABLE, OPTIMIZE TABLE et REPAIR TABLE provoquent une validation implicite.

QUESTIONS D'AUTO-EVALUATION

2. La commande _____ met également fin à une transaction si des tables sont actuellement verrouillées.

V. SYNTAXE DE SAVEPOINT ET ROLLBACK TO SAVEPOINT

<i>SAVEPOINT identifiant</i> <i>ROLLBACK TO SAVEPOINT identifiant</i>
--

Depuis MySQL 4.0.14 et 4.1.1, InnoDB supporte les instructions SQL SAVEPOINT et ROLLBACK TO SAVEPOINT.

L'instruction **SAVEPOINT** définit un point d'enregistrement de transaction nommé avec **un nom d'identificateur**. Si la transaction en cours a déjà un point de sauvegarde portant le même nom, l'ancien point de sauvegarde est supprimé et un nouveau est défini.

L'instruction **ROLLBACK TO SAVEPOINT** annule une transaction sur le point de sauvegarde **nommé**. Les modifications apportées à la transaction en cours sur les lignes après la définition du point de sauvegarde sont annulées dans la restauration, mais InnoDB ne libère pas les verrous de ligne stockés en mémoire après le point de sauvegarde. (Notez que pour une nouvelle ligne insérée,

les informations de verrouillage sont portées par l'ID de transaction stockée dans la ligne; le verrou n'est pas stocké séparément en mémoire. Dans ce cas, le verrou de ligne est libéré à l'annulation.) Points de sauvegarde définis plus tard, le point de sauvegarde nommé est supprimé.

Si l'instruction renvoie l'erreur suivante, cela signifie qu'aucun point de sauvegarde portant le nom spécifié n'existe:

ERREUR 1181: Erreur 153 lors du ROLLBACK

Tous les points de sauvegarde de la transaction en cours sont supprimés si vous exécutez un COMMIT ou un ROLLBACK qui ne nomme pas un point de sauvegarde.

QUESTIONS D'AUTO-EVALUATION

3. L'instruction _____ définit un point d'enregistrement de transaction nommé avec un nom d'identificateur.

VI. SYNTAXE DES TABLES DE VERROUILLAGE ET DE VERROUILLAGE

```
LOCK TABLES
tbl_name [AS alias] {READ [LOCAL] | [LOW_PRIORITY] WRITE}
[, tbl_name [AS alias] {READ [LOCAL] | [LOW_PRIORITY] WRITE}]
...
```

LOCK TABLES verrouille les tables pour le thread actuel. **UNLOCK TABLES** libère tous les verrous détenus par le thread actuel. Toutes les tables verrouillées par le thread actuel sont implicitement déverrouillées lorsque le thread émet une autre TABLES DE VERROUILLAGE ou lorsque la connexion au serveur est fermée.

Remarque: **LOCK TABLES** n'est pas sécurisé pour les transactions et valide implicitement toutes les transactions actives avant de tenter de verrouiller les tables.

Depuis **MySQL 4.0.2**, pour utiliser **LOCK TABLES**, vous devez disposer du privilège global **LOCK TABLES** et d'un privilège **SELECT** sur les tables impliquées. Dans **MySQL 3.23**, vous devez disposer des privilèges **SELECT**, **INSERT**, **DELETE** et **UPDATE** pour les tables.

Les principales raisons d'utiliser les tables de verrouillage sont pour émuler des transactions ou pour obtenir plus de rapidité lors de la mise à jour des tables.

Si un thread obtient un verrou **READ** sur une table, ce thread (et tous les autres threads) ne peut lire que dans la table. Si un thread obtient un verrou **WRITE** sur une table, seul le thread qui détient le verrou peut lire ou écrire dans la table. Les autres threads sont bloqués.

La différence entre **READ LOCAL** et **READ** est que **READ LOCAL** permet l'exécution d'instructions **INSERT** non conflictuelles tant que le verrou est maintenu. Cependant, cela ne peut pas être utilisé si vous allez manipuler les fichiers de base de données en dehors de MySQL pendant que vous maintenez le verrou.

Lorsque vous utilisez **LOCK TABLES**, vous devez verrouiller toutes les tables que vous allez utiliser dans vos requêtes.

Si vous utilisez une table plusieurs fois dans une requête (avec des alias), vous devez obtenir un verrou pour chaque alias. Tant que les verrous obtenus avec une instruction LOCK TABLES sont en vigueur, vous ne pouvez accéder à aucune table non verrouillée par cette instruction.

Si vos requêtes font référence à une table utilisant un alias, vous devez alors la verrouiller à l'aide de ce même alias. Cela ne fonctionnera pas de verrouiller la table sans spécifier l'alias:

APPLICATION :

Nous allons verrouiller la table **bb** sans verrouiller son alliasse :

```
mysql> LOCK TABLE bb READ;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM bb AS monAlliasse;
ERROR 1100 (HY000): Table 'monAlliasse' was not locked with LOCK TABLES
mysql>
mysql>
mysql> SELECT * FROM bb;
+-----+-----+
| id    | recaptulatif |
+-----+-----+
| 1     | 11200000     |
| 2     | 31800000     |
| 3     | 37540500     |
| 4     | 41400350     |
| 5     | 25000000     |
+-----+-----+
5 rows in set (0.00 sec)

mysql> insert into bb (id) values (6);
ERROR 1099 (HY000): Table 'bb' was locked with a READ lock and can't be updated
```

À l'inverse, si vous verrouillez une table à l'aide d'un alias, vous devez y faire référence dans vos requêtes utilisant cet alias:

```
mysql> LOCK TABLE bb AS monAlliasse READ;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM bb;
ERROR 1100 (HY000): Table 'bb' was not locked with LOCK TABLES
mysql>
```

Les verrous **WRITE** ont normalement une priorité plus élevée que les verrous **READ**, afin de garantir que les mises à jour sont traitées le plus rapidement possible. Cela signifie que si un thread obtient un verrou **READ** et qu'un autre thread demande un verrou **WRITE**, les demandes de verrou **READ** suivantes attendent que le thread **WRITE** obtienne le verrou et le libère. Vous pouvez utiliser les verrous **LOW_PRIORITY WRITE** pour permettre à d'autres threads d'obtenir des verrous **READ** pendant que le thread attend le verrou **WRITE**.

NB :

Vous ne devez utiliser les verrous **LOW_PRIORITY WRITE** que si vous êtes certain qu'il y aura un moment où aucun thread n'aura un verrou **READ**.

LOCK TABLES fonctionne comme suit:

1. Triez toutes les tables à verrouiller dans un ordre défini en interne. Du point de vue de l'utilisateur, cet ordre n'est pas défini.
2. Si une table est verrouillée avec un verrou en lecture et un verrou en écriture, mettez le verrou en écriture avant le verrou en lecture.
3. Verrouillez une table à la fois jusqu'à ce que le thread obtienne tous les verrous.

Si vous allez exécuter de nombreuses opérations sur un ensemble de tables, il est beaucoup plus rapide de verrouiller les tables que vous allez utiliser. L'inconvénient est qu'aucun thread ne peut mettre à jour une table verrouillée en lecture (y compris celle qui contient le verrou) et qu'aucun thread ne peut lire une table verrouillée en écriture autre que celle qui contient le verrou.

La raison pour laquelle certaines opérations sont plus rapides sous **LOCK TABLES** est que **MySQL** ne vide pas le cache de clés pour les tables verrouillées jusqu'à ce qu'**UNLOCK TABLES** soit appelé. Normalement, le cache de clé est vidé après chaque instruction SQL. Le verrouillage des tables MyISAM accélère leur insertion, leur mise à jour ou leur suppression.

Si vous utilisez un moteur de stockage dans **MySQL** qui ne prend pas en charge les transactions, vous devez utiliser **LOCK TABLES** si vous souhaitez vous assurer qu'aucun autre thread ne se situe entre un **SELECT** et une **UPDATE**. L'exemple présenté ici nécessite **LOCK TABLES** pour s'exécuter en toute sécurité:

```
mysql> LOCK TABLES trans READ, customer WRITE;
mysql> SELECT SUM(value) FROM trans WHERE customer_id=some_id;
mysql> UPDATE customer
-> SET total_value=sum_from_previous_statement
-> WHERE customer_id=some_id;
mysql> UNLOCK TABLES;
```

Sans **LOCK TABLES**, il est possible qu'un autre thread insère une nouvelle ligne dans la table de conversion entre l'exécution des instructions **SELECT** et **UPDATE**.

Vous pouvez éviter d'utiliser LOCK TABLES dans de nombreux cas en utilisant des mises à jour relatives (**UPDATE** customer SET valeur = valeur + nouvelle_valeur) ou la fonction **LAST_INSERT_ID ()**.

Vous pouvez également éviter de verrouiller les tables dans certains cas en utilisant les fonctions de verrouillage d'avis au niveau utilisateur **GET_LOCK ()** et **RELEASE_LOCK ()**. Ces verrous sont enregistrés dans une table de **hachage** sur le serveur et implémentés avec pthread_mutex_lock () et pthread_mutex_unlock () pour une vitesse élevée.

Vous pouvez verrouiller toutes les tables de toutes les bases de données avec des verrous de lecture avec la commande **FLUSH TABLES WITH READ.**

Déclaration LOCK. C'est un moyen très pratique d'obtenir des sauvegardes si vous disposez d'un système de fichiers tel que Veritas qui peut prendre des instantanés à temps.

REMARQUE: Si vous utilisez **ALTER TABLE** sur une table verrouillée, celle-ci peut être déverrouillée.

QUESTIONS D'AUTO-EVALUATION

4. La commande _____ libère tous les verrous détenus par le thread actuel.
5. Si une table est verrouillée avec un verrou en lecture et en écriture, placez le verrou _____ avant le verrou _____.

VII. SYNTAXE SET TRANSACTION

```
SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL  
{READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ |  
SERIALIZABLE}
```

Cette instruction définit le niveau d'isolation de transaction pour la transaction suivante, globalement ou pour la session en cours.

- ❖ Le comportement par défaut de **SET TRANSACTION** consiste à définir le niveau d'isolation pour la prochaine transaction (pas encore démarrée).

- ❖ Si vous utilisez le mot clé **GLOBAL**, l'instruction définit le niveau de transaction par défaut pour toutes les nouvelles connexions créées à partir de ce moment. Les connexions existantes ne sont pas affectées. Vous avez besoin du privilège **SUPER** pour le faire.
- ❖ L'utilisation du mot-clé **SESSION** définit le niveau de transaction par défaut pour toutes les transactions futures effectuées sur la connexion en cours.
- ❖ Vous pouvez définir le niveau d'isolation global par défaut initial pour **MySQLD** avec l'option **--transaction- isolation**.
- ❖ Pour définir le niveau global d'isolation par défaut au démarrage du serveur, utilisez l'option **--transaction-isolation = level** sur **mysqld** sur la ligne de commande ou dans un fichier d'options.
- ❖ Les valeurs de niveau pour cette option utilisent des tirets plutôt que des espaces, les valeurs autorisées sont donc **READUNCOMMITTED**, **READ-COMMITTED**, **REPEATABLE-READ** ou **SERIALIZABLE**.

PAR EXEMPLE, pour définir le niveau d'isolation par défaut sur **REPEATABLE READ**, utilisez ces lignes dans la section **[mysqld]** d'un fichier d'options:

```
[mysqld]
transaction-isolation = REPEATABLE-READ
```

Mysqld se trouve dans le dossier de configuration MySQL **my.ini**.

Pour déterminer les niveaux d'isolation global et de transaction de session au moment de l'exécution, vérifiez la valeur de la variable système **tx_isolation**:

```
SELECT @@GLOBAL.tx_isolation, @@tx_isolation;
```

```
mysql> select @@GLOBAL.tx_isolation, @@tx_isolation;
+-----+-----+
| @@GLOBAL.tx_isolation | @@tx_isolation |
+-----+-----+
| REPEATABLE-READ      | REPEATABLE-READ |
+-----+-----+
1 row in set (0.00 sec)
```

La liste suivante décrit comment MySQL prend en charge les différents niveaux de transaction:

READ UNCOMMITTED

Les instructions **SELECT** sont exécutées de manière non verrouillable, mais une version antérieure possible d'une ligne peut être utilisée. Ainsi, en utilisant ce niveau d'isolation, ces lectures ne sont pas cohérentes. Cela s'appelle également une lecture "mensuelle". Sinon, ce niveau d'isolement fonctionne comme **READ COMMITTED**.

READ COMMITTED

Un niveau d'isolation quelque peu similaire à Oracle en ce qui concerne les lectures cohérentes (non verrouillables): chaque lecture cohérente, même au sein d'une même transaction, définit et lit son propre instantané récent.

Pour le verrouillage des lectures (SELECT avec FOR UPDATE ou LOCK IN SHARE MODE), InnoDB verrouille uniquement les enregistrements d'index, pas les espaces vides qui les précèdent, et permet ainsi l'insertion libre de nouveaux enregistrements à côté des enregistrements verrouillés. Pour les instructions UPDATE et DELETE, le verrouillage varie selon que l'instruction utilise un index unique avec une condition de recherche unique (telle que WHERE id = 100) ou une condition de recherche du type de plage (telle que WHERE id > 100). Pour un index unique avec une condition de recherche unique, InnoDB verrouille uniquement l'enregistrement d'index trouvé, pas l'écart qui le précède. Pour les recherches par type de plage, InnoDB verrouille la plage d'index analysée à l'aide de verrous d'espacement ou de serrures nextkey (espace plus enregistrement-index) pour bloquer les insertions d'autres sessions dans les espaces couverts par l'intervalle. Cela est nécessaire car les «lignes fantômes» doivent être bloquées pour que la réplication et la récupération MySQL fonctionnent.

REPEATABLE-READ

C'est le niveau d'isolation par défaut pour **InnoDB**. Pour les lectures cohérentes, il existe une différence importante par rapport au niveau d'isolation **READ COMMITTED**: toutes les lectures cohérentes d'une même transaction lisent l'instantané établi par la première lecture. Cette convention signifie que si vous émettez plusieurs instructions SELECT simples (non verrouillables) au sein d'une même transaction, ces instructions SELECT sont également cohérentes.

Pour le verrouillage des lectures (instructions SELECT, FOR UPDATE ou LOCK IN SHARE MODE), UPDATE et DELETE, le verrouillage dépend du fait que l'instruction utilise un index unique avec une condition de recherche unique ou une condition de recherche de type plage. Pour un index unique avec une condition de recherche unique, InnoDB verrouille uniquement l'enregistrement d'index trouvé, pas l'écart qui le précède. Pour les autres conditions de recherche, InnoDB verrouille la plage d'index analysée à l'aide de verrous d'espacement ou de serrures de clé suivante (espace plus enregistrement d'index) pour bloquer les insertions d'autres sessions dans les espaces couverts par la plage.

SERIALIZABLE

Ce niveau est similaire à **REPEATABLE READ**, mais InnoDB convertit implicitement toutes les instructions **SELECT** simples en **SELECT ... LOCK IN SHARE MODE** si l'auto-validation est

désactivée. Si la validation automatique est activée, le SELECT est sa propre transaction. Par conséquent, il est connu pour être en lecture seule et peut être sérialisé s'il est exécuté en tant que lecture cohérente (non verrouillante) et n'a pas besoin d'être bloqué pour d'autres transactions. (Cela signifie que pour forcer un simple SELECT à bloquer si d'autres transactions ont modifié les lignes sélectionnées, vous devez désactiver autocommit.).

QUESTIONS D'AUTO-EVALUATION

6. L'utilisation du mot clé _____ permet de définir le niveau de transaction par défaut pour toutes les transactions futures effectuées sur la connexion en cours.

RESUME

Cette unité couvre les aspects du traitement des transactions de MySQL.

1. Syntaxe de démarrage des transactions, des validations et des restaurations: Cette rubrique fournit la syntaxe et des exemples de Démarrage d'une transaction, validation des modifications apportées au cours d'une transaction et annulation des modifications non souhaitées apportées au cours d'une transaction.
2. Déclarations qui ne peuvent pas être annulées: cette rubrique traite de la théorie qui sous-tend la gestion des transactions qui ne peuvent pas être annulées.
3. Instructions entraînant une validation implicite: cette rubrique répertorie les instructions SQL entraînant une validation implicite dans les bases de données.
4. Syntaxe du point de sauvegarde et de la restauration vers le point de sauvegarde: L'instruction SAVEPOINT définit un point de sauvegarde de transaction nommé avec un nom d'identificateur. L'instruction ROLLBACK TO SAVEPOINT annule une transaction sur le point de sauvegarde nommé.
5. Verrouiller les tables et déverrouiller les tables Syntaxe: LOCK TABLES verrouille les tables pour le thread actuel. UNLOCK TABLES libère tous les verrous détenus par le thread actuel. Cette rubrique fournit également des exemples appropriés pour la compréhension de concepts.
6. Définir la syntaxe de la transaction: Cette instruction définit le niveau d'isolation de la transaction pour la transaction suivante, globalement ou pour la session en cours. Le comportement par défaut de SET TRANSACTION consiste à définir le niveau d'isolation pour la prochaine transaction (pas encore démarrée).

QUESTIONS SUR LES TERMINAUX

1. Discutez de certaines des déclarations transactionnelles qui provoquent une validation implicite.
2. Décrivez les syntaxes de Savepoint et Rollback to Savepoint.

FIN !